Blackjack Analytics API – Technical Report

A brief report containing technical information regarding the Blackjack Analytics API. Topics: System architecture, technology stack justification, API design, database design, testing approaches, challenges and solutions, limitations and improvements.

GitHub Repository: https://github.com/AlexFairleyLU/Blackjack-Analytics-API

API Documentation: https://github.com/AlexFairleyLU/Blackjack-Analytics-API/blob/main/api_documentation.pdf

Presentation: …

By Alexander Fairley [sc22af2] @ University of Leeds

Contents

Introduction

This project implements a RESTful API for recording and analysing blackjack gameplay data. The system allows users to create sessions, record individual hands, and compute statistical analytics such as win rates and betting performance. In addition, the API incorporates a blackjack basic strategy dataset to evaluate player decisions and provide strategy recommendations.

System Architecture

The system is designed using a layered architecture that separates core functionality. Request handling, system logic and data retention are handled individually by their respective packages. This approach helps to improve the maintainability and scalability of the program as well as the transparency of code operation. This is particularly important considering this is an initial application design, with high potential for future expansion.

Modelled at a higher level, the architecture can be treated as having four main layers:

Client → API Routes → Data Access → Database

1. Client layer – The client interacts with the API through structured HTTP requests. The layout of available requests and responses are pre-defined by Pydantic schemas.
2. API Routes layer – All HTTP requests made by the client are handled by API routes implemented with FastAPI. When a request is received, the route first validates the input against the expected schema definition. It then executes specified pre-response logic, with database interaction handled by SQLAlchemy, before returning structured JSON responses.
3. Data Access – SQLAlchemy is used to implement a variety of database operations. This allows the application to interact with the database: adding, updating, removing and fetching data. These interactions are done through Python objects rather than raw SQL queries which improves readability and reduces the likelihood of errors or vulnerabilities.
4. Database layer – The relational database has been implemented using PostgreSQL. The model is designed around the core features of the system: users, sessions, hands and basic strategy. These entities are linked through well-defined relationships for efficient querying, as can be seen by the models specified in the repository.

In addition to how the program is composed, the repository follows a conventional structure for API programs. Within the *app/* directory, several folders separate key responsibilities into distinct components:

- models/: Defines the database using SQLAlchemy ORM models
- routes/: Defines all API endpoints and handles HTTP request/response logic
- schemas/: Contains data validation models using Pydantic, ensuring all incoming and outgoing data conforms to the designated format
- scripts/: Contains a collection of scripts for database seeding and recreation

The system also benefits from automatically generated API documentation provided by Swagger UI, which reflects the structure of the routes and schemas. This enhances usability by allowing developers to explore and test endpoints interactively without additional configuration.

Technology Stack

Programming Language – **Python:**

Python has a simple syntax, is easy to use and is demonstrated often in university coding examples, both in web services and other modules. In addition, it has specific advantages when it comes to web development: widely used and available frameworks, easy database interaction/connectivity, simple debugging and straightforward deployment options (Geeksforgeeks, 2025). The coupling of these factors made it an obvious choice as the language for this project.

Web Framework – **Fast API:**

As a beginner in both web services and designing web APIs, Fast API was a great choice. It was simple to learn/implement, and its many features were very helpful in the design of this project: Pydantic for data validation, seamless integration with uvicorn (an ASGI server) and

in-built SwaggerUI documentation made the program more functional and professional (Masioli, 2025).

ORM – **SQLAlchemy:**

For Object-Relational Mapping, SQLAlchemy is one of the most popular packages used in Python. It provides a simple API for interacting with a database through objects, rather than individual SQL queries (Datta, 2024). It's variety of available interactive methods, including numerical functions, make it a perfect tool for this project.

Database – **PostgreSQL:**

For this project, working with an SQL database was required. PostgreSQL was recommended to me due to its relational model, ACID compliance and support of complex SQL queries (Hemant, 2025), making it an exceptional tool for use in this project.

Testing Approach

Due to the informed decisions made when deciding the technology stack, testing for this project was made very simple. SwaggerUI provides a fantastic interface for testing the web API, ensuring requests and responses were as expected. Additionally, unique seeding scripts were developed to populate the database with test data, which could then be used to repeatably verify database interaction and results.

Challenges and Solution

1. Complex analytics - A core feature of this project is the analytical data provided by the API to user's regarding their performance and success. Calculating these statistics using database values can lead to quite a lot of overhead, mainly from excessive database interaction and reductive calculations. Having also not used SQLAlchemy before, I was concerned that this could be an area of poor performance. However, through research and creative AI usage, I was able to explore SQL aggregation functions that the package provides, streamlining this process in an efficient manner.
2. Seeding strategy dataset – To introduce the basic strategy data, a seed file needed to be created. In my initial efforts, the seed file was excessively long, as I manually encoded each strategy for separate hands. Upon review and research, I realised it would be far more practical to instead use the rules that basic strategy is derived from to populate the dataset, reducing the file size and complexity, while improving comprehension.

Areas for Improvement

- Lack of administration – The program currently lacks administrative routes or endpoints, which means that without manual database access it is impossible to control or observe all stored information. It would be a good idea to implement such a feature, to improve security and control over program use and data storage.
- Lack of data usage restrictions – As this is a prototype of the API, users are not currently prevented from creating an excessive number of sessions or hands. This could be a potential issue if the API encountered a malicious client. To address this, it

would be prudent to limit the total number of sessions/hands a single user can create, or storing older (unlikely to be accessed) sessions as compressed data instead.
- Strategy limitations – Currently, the strategy model is based upon generalised basic Blackjack strategy. However, this strategy can change considerably due to several factors, such as casino rules, number of decks used or if the player is counting cards. It would make the API far more complex and expansive if all these considerations were accounted for in the dataset.
- Expected value analysis – A key field for gambling is expected value, the amount of money someone is expected to win/lose over the duration of a session, based on their strategy. Expanding the current analytic tools to incorporate expected value and other predictive components would further improve the quality of the project.

Generative AI

Declaration – I acknowledge the use of ChatGPT as a tool in the development of this project. ChatGPT was chosen over Copilot due to familiarity with the software, and appropriate measures were taken to ensure that no university policies regarding the use of AI were breached. Examples of use include:

- Structure advice – "Should all routes defined in the routes folder have associated schema?": **Response –** Advised me that not all routes require schemas but should when the route accepts input or returns structured data. This prompted me to add schemas for both the analytics and strategy routes. Asking GenAI for advice on structuring and general industry standards for development aided in ensuring the project was to a professional level.
- Verification of functionality – "Lets verify all endpoints have the correct HTTP status codes": **Response –** Informed me of the correct endpoints for implemented CRUD routes so that I could cross-reference with my program and correct where necessary. As someone who is new to API design, the ability to query with the AI to see if my program performs correctly is very helpful guidance.
- Research – "Can you give me examples of how a RESTful API can be used and how a GraphQL API can be used": **Response –** Broke down multiple use-cases for each and how they differ. This was very helpful in making quick but informed design choices.
- Debugging – "After my previous changes, I now see this error…": **Response** – Having used GenAI to aid in debugging at multiple points in this project, it is a fantastic tool at swiftly identifying and isolating bugs. In cases like the above, it provides meaningful steps to identify the cause, or identifies it based off purely the provided error message.
- Code examples – "One thing that I think would be useful is some way to pre-initialise the database with test data, so instead of testing manually each time I can test it against the preset data": **Response –** Provided both examples of code and a description of its functionality. This was particularly useful when trying to generate seed files for the test data.

Example GenAI Chat Logs

1. Should all routes defined in the routes folder have associated schema?
   Response: Short answer: not all routes must have schemas — but most should, and using them consistently improves marks. Let's break it down properly.

   ✅ When You SHOULD Use Schemas:
   You should use Pydantic schemas whenever:
   1. The route accepts input (request body) Example: POST /users, POST /sessions/{session_id}/hands, PUT /users/{user_id}

   👉 These must have schemas: def create_user(user: UserCreate).
   Why: validates input, prevents invalid data, improves Swagger documentation.
   2, The route returns structured data. Example: GET /users/{user_id}, GET/sessions/{session_id}, GET /analytics/session/{session_id}

   👉 You should define response schemas: @router.get("/{id}", response_model=UserResponse).
   Why: enforces consistent output, improves documentation, looks more "professional" to marker
2. Let's verify all endpoints have the correct HTTP status codes. **Response:** Good step — this is exactly the kind of small but important polish markers look for. APIs that return correct HTTP status codes demonstrate professional REST design. Since you're using FastAPI, most responses default to 200 OK, so we mainly need to ensure:

   Create endpoints → 201, Delete endpoints → 204, Not found errors → 404, Invalid input → 400, Successful reads → 200...

References

Datta, D. 2024. What is SQLAlchemy Used For? An Overview with Practical Examples. [Online]. [Accessed 19 March 2026]. Available from: https://www.cdata.com/blog/what-is-sqlalchemy

Geeksforgeeks. 2025. Why to Use Python For Web Development? [Online]. [Accessed 19 March 2026]. Available from: https://www.geeksforgeeks.org/python/why-to-use-python-for-web-development/

Hemant. 2025. 10 Reasons Why PostgreSQL is a Game-Changer for Developers. [Online]. [Accessed 19 March 2026]. Available from: https://hemant656.medium.com/10-reasons-why-postgresql-is-a-game-changer-for-developers-e3e5d8054030

Masioli, A. 2025. Why FastAPI is the Right Choice for Building Modern APIs: A Simple Example for Beginners. [Online]. [Accessed 19 March 2026]. Available from: https://medium.com/@adelino-masioli/why-fastapi-is-the-right-choice-for-building-modern-apis-a-simple-example-for-beginners-5fdcc9d82333